

Understanding Mathematical Concepts via Go Language

Graph (Part 1)

Hyosang Kang (DGIST)

고 언어로 수학적 개념 이해하기

그래프 (파트 1)

강효상 (DGIST)

Graphs

Function

$$f : D \rightarrow R; \quad x \mapsto f(x)$$

- D : Domain

```
type Region interface {  
    Set[Point]  
    Corner() Vector  
    Lengths() []Real  
}
```

- $f(x)$: Relation

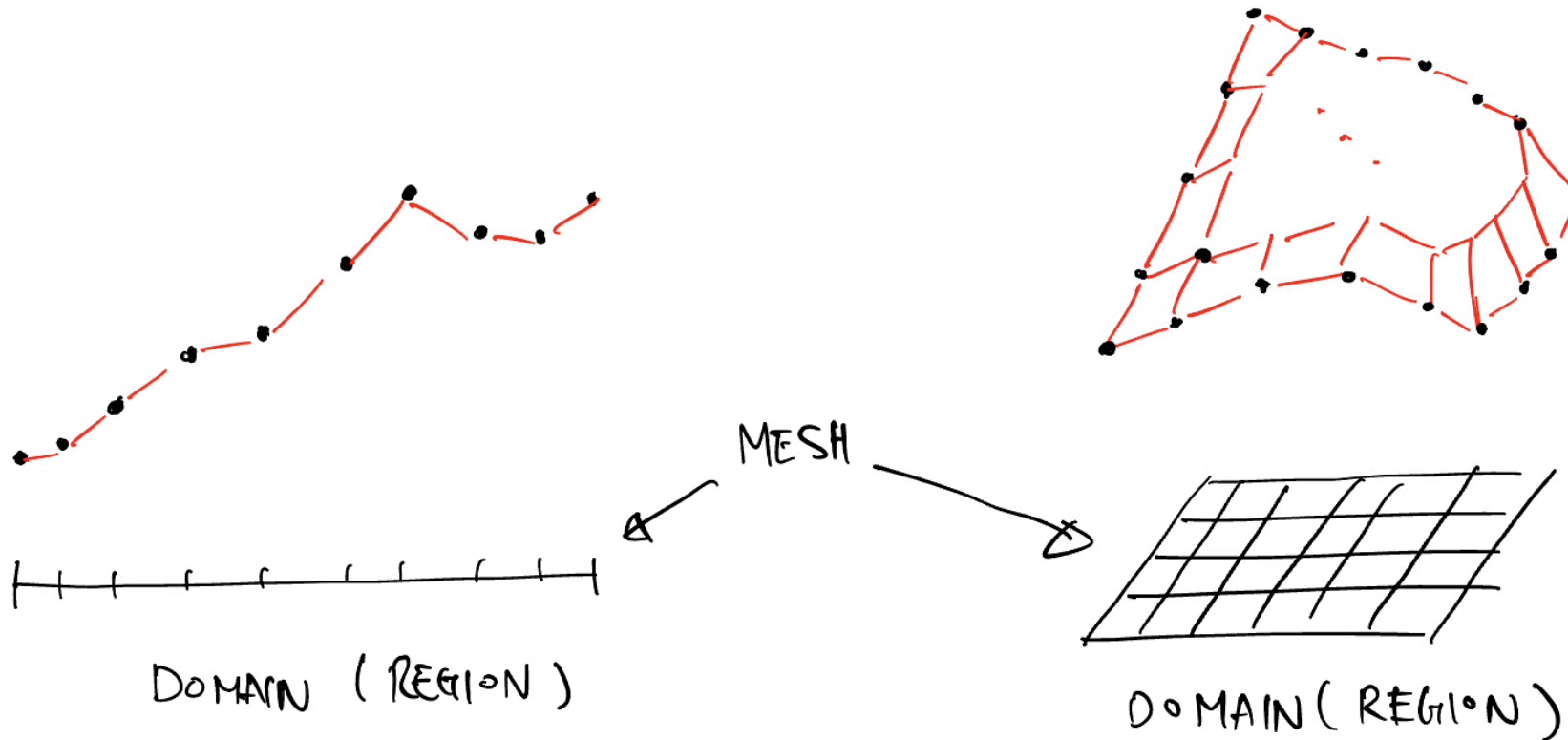
```
type Map[T1, T2 any] interface {  
    Map(T1) T2  
}
```

Graph

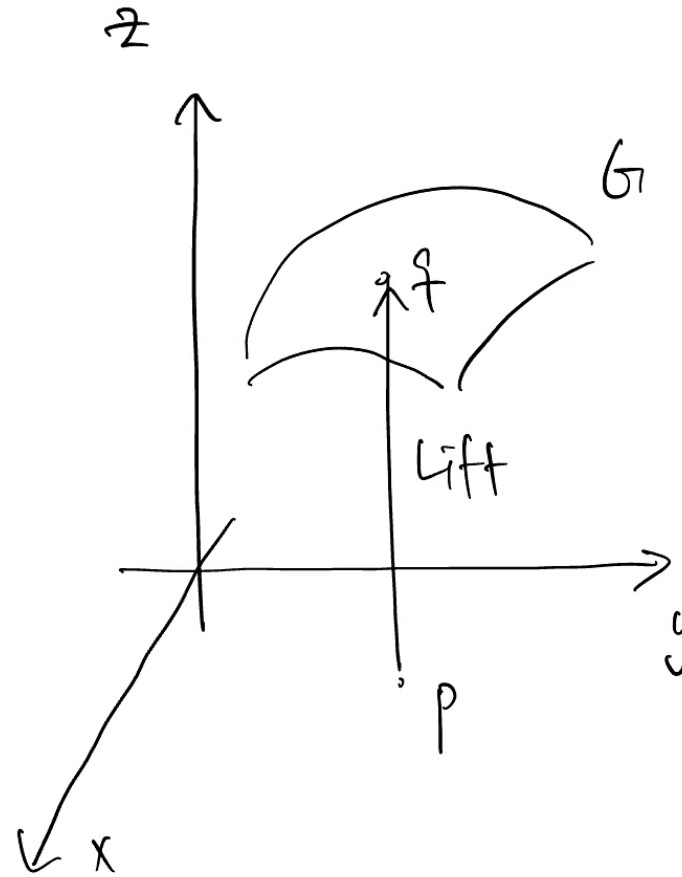
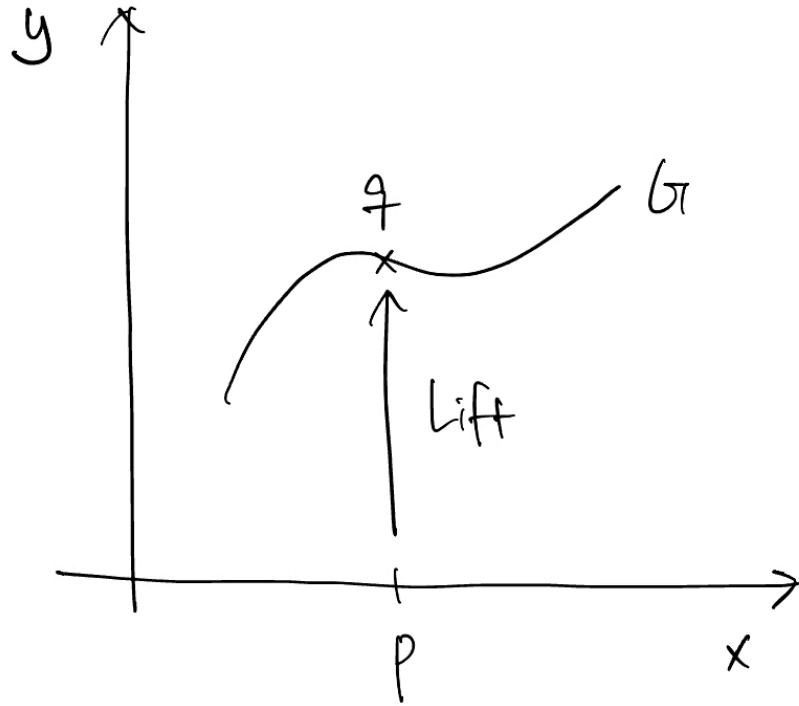
$$G = \{(x, f(x)) \mid x \in D\}$$

- We can visualize the graph only on the 2D plane.
- 2D plane can show the 1-dimensional line effectively.
- We can consider graph as a collection lines.

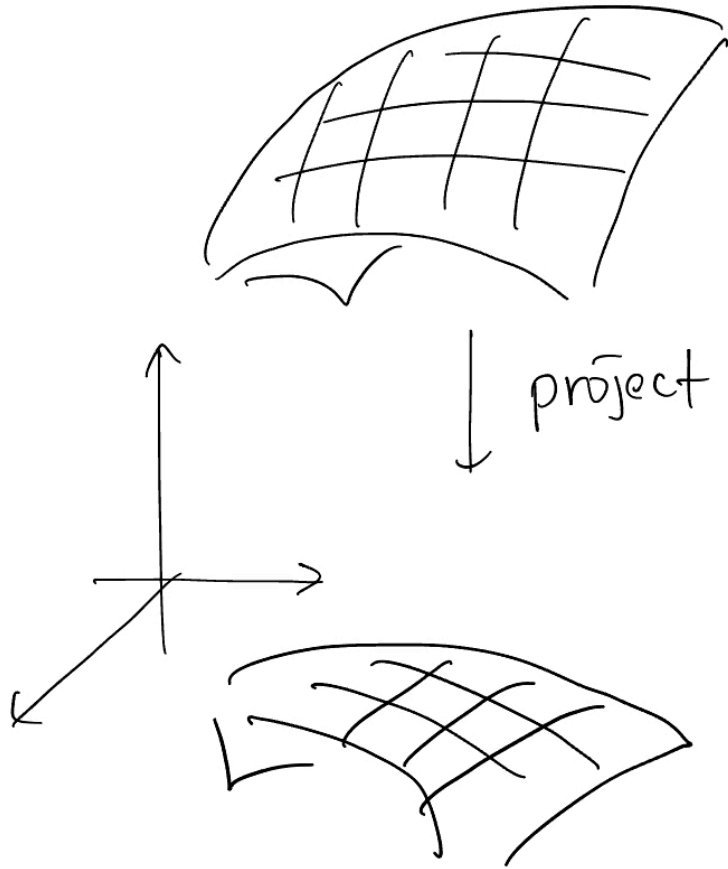
How to construct a graph



What we really need



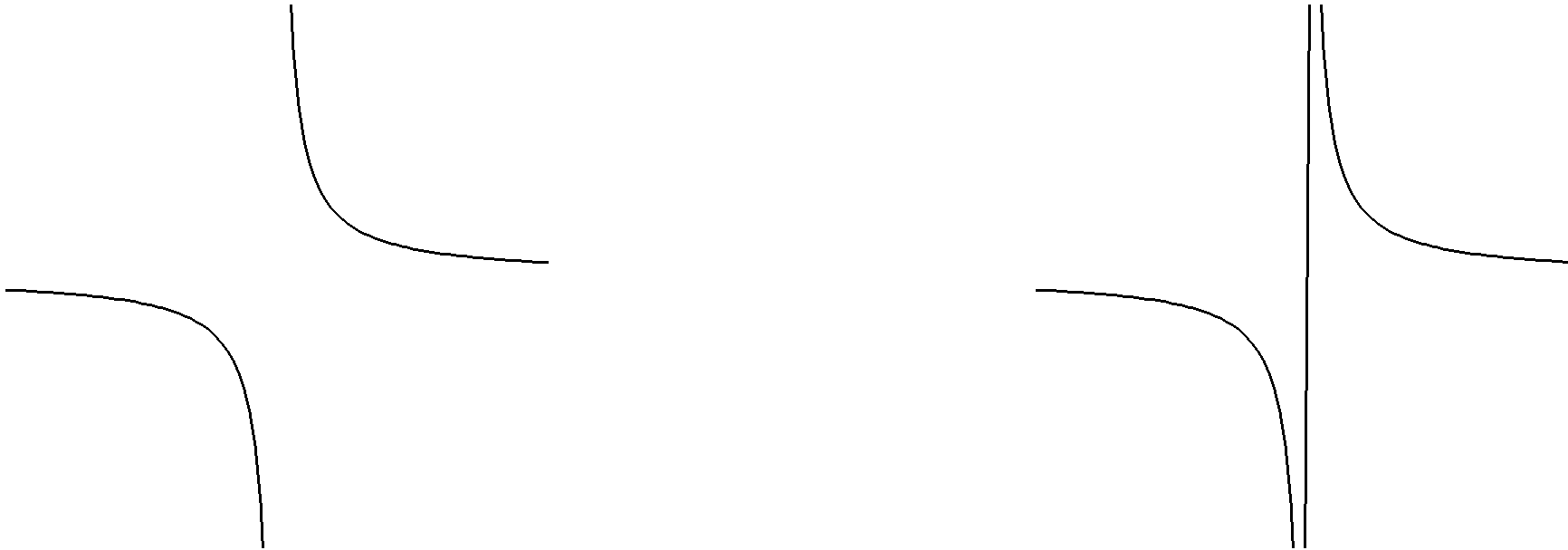
How to visualize a graph



```
type Grapher interface {  
    Region  
    Lift(Point) Point  
    Project(Point) Point  
}
```


How to manage discontinuity(or continuity)

$$f(x) = \frac{1}{x}$$



```
type Continuous interface {  
    Map[Point, Vector]  
    Eps(Real, Point) Real  
}
```

Graph

```
func Graph(gpr Grapher, nsub int) <-chan [2]Point {  
    dim := gpr.Corner().Len()  
    mesh := Mesh(gpr, nsub)  
    ch := make(chan [2]Point)  
    go func() {  
        defer close(ch)  
        for i, p := range mesh {  
            for _, j := range Neighbors(i, nsub, dim) {
```

```

    q := mesh[j]
    line := [2]Point{gpr.Lift(p), gpr.Lift(q)}
    if fn, ok := gpr.(Continuous); ok {
        delta := Distance(p, q)
        eps := fn.Eps(delta, p)
        if eps.GreaterThan(Distance(fn.Map(p), fn.Map(q))) {
            ch <- line
        }
    } else {
        ch <- line
    }
}

}

}()
return ch
}

```

Ingredients for the output

```
type GraphOption struct {  
    Nsub                int  
    Xmin, Xmax, Ymin, Ymax float64  
    Width, Height       int  
    Filename            string  
}
```

Save to image

```
func (opt GraphOption) Save(gpr Grapher) {  
    img := image.NewPaletted(  
        image.Rect(0, 0, opt.Width, opt.Height),  
        []color.Color{  
            color.White,  
            color.Black,  
        })  
    for line := range Graph(gpr, opt.Nsub) {  
        opt.DrawLine(img,  
            gpr.Project(line[0]),  
            gpr.Project(line[1]))  
    }  
    fp, _ := os.Create(opt.Filename)  
    defer fp.Close()  
    png.Encode(fp, img)  
}
```

Drawing lines

```
func (opt GraphOption) DrawLine(img *image.Paletted, p, q Point) {  
    x0, y0 := p.Map(0), p.Map(1)  
    x1, y1 := q.Map(0), q.Map(1)  
    i0, j0 := opt.Pixel(x0.ToFloat(), y0.ToFloat())  
    i1, j1 := opt.Pixel(x1.ToFloat(), y1.ToFloat())  
    DrawPoint := func(i, j int) {  
        for l := -1; l <= 1; l++ {  
            img.Set(i+l, j, color.Black)  
            img.Set(i, j+l, color.Black)  
        }  
    }  
}
```

```
n := Max(Abs(i0-i1), Abs(j0-j1))
if n == 0 {
    DrawPoint(i0, j0)
} else {
    is := Linspace(i0, i1, n)
    js := Linspace(j0, j1, n)
    for k := 0; k <= int(n); k++ {
        i, j := is[k], js[k]
        DrawPoint(i, j)
    }
}
```

Cartesian to Pixel

```
func (opt GraphOption) Pixel(x, y float64) (int, int) {  
    dw := (opt.Xmax - opt.Xmin) / float64(opt.Width)  
    dh := (opt.Ymax - opt.Ymin) / float64(opt.Height)  
    i := int((x - opt.Xmin) / dw)  
    j := int((opt.Ymax - y) / dh)  
    return i, j  
}
```


Subdividing between to numbers

```
func Linspace[T int | float64](a, b T, n int) []T {  
    if n <= 0 {  
        return nil  
    }  
    af, bf := float64(a), float64(b)  
    var h float64 = (bf - af) / float64(n)  
    var arr []T  
    for i := 0; i <= n; i++ {  
        arr = append(arr, T(af+float64(i)*h))  
    }  
    return arr  
}
```

Other utilities

```
func Max[T int | float64](x, y T) T {  
    if x < y {  
        return y  
    }  
    return x  
}  
  
func Abs[T int | float64](x T) T {  
    if x < 0 {  
        return -x  
    }  
    return x  
}
```

This video was produced with the support of CTL in DGIST.

Production: Hyosang Kang

Editing: Jongwoo Baek

본 동영상은 DGIST의 CTL의 지원을 받아 제작되었습니다.

제작: 강효상

편집: 백종우